

DeepSeek Harness 的工具插件化设计

2.6.1 DeepSeek Harness 是什么？

DeepSeek Harness 是 DeepSeek AI 开源的一套 Agent Harness，目前处于 developer preview。它不是一个新模型，也不是 Function Calling API，而是运行 Agent 的工程框架。

可以先把 Harness 理解成 Agent 的「运行外壳」：

模型	Harness
决定下一步想做什么	提供当前能做什么，并控制这一步怎样执行

2.6.2 它与 pi 和常见 Agent Harness 有什么区别？

比较维度	pi 0.83.0	DeepSeek Harness	常见的应用级 Agent 框架
主要关注点	轻量、可组合的模型调用与 Agent Loop	插件优先的 Agent 运行平台	快速搭建固定 Workflow 或 Agent 应用
工具扩展	registerTool() 注册工具，使用执行前后事件扩展行为	插件通过 ctx.tools Service 注册工具	通常在初始化时传入工具列表或节点
核心 Loop	源码集中，适合直接理解调用和回传顺序	运行能力通过插件与事件协作，强调可替换和装配	Loop 或 Workflow 通常由框架预先定义
依赖处理	扩展使用宿主提供的 API 和上下文	插件声明所需 Service，依赖未就绪时等待加载	多由应用启动代码手工保证初始化顺序
卸载与清理	扩展跟随会话或宿主生命周期管理	注册行为归属插件 Fiber，卸载时自动撤销	经常只考虑启动和注册，较少讨论动态卸载
最值得学习的部分	Agent Loop、Tool Call、Tool Result 和 Hook 顺序	Service、依赖、生命周期和可撤销注册	业务流程建模与快速交付

DeepSeek Harness 的工具插件化设计

2.6.3 四个新概念

概念	先把它理解成什么	解决什么问题
Registry	按名称保存工具定义的索引	Tool Call 到来时，Runtime 到哪里寻找工具
Plugin	向 Harness 贡献一种能力的自包含模块	新增工具或治理能力时，怎样避免修改核心 Loop
Service	Harness 提供给插件使用的具名运行能力	插件怎样获得 Tool Runtime、LLM 或 Agent 能力
Lifecycle	插件从等待、加载、运行到卸载的过程	依赖未就绪怎么办，卸载时由谁清理资源



真正要学习的四个 Harness 机制

- 1 工具定义与 Agent Loop 分离，新增工具不修改循环；
- 2 插件通过 Service 获取能力，不直接访问 Harness 内部实现；
- 3 注册行为必须可撤销，谁注册就由谁负责清理；
- 4 权限、Sandbox、超时和 Trace 通过稳定扩展点覆盖多类工具。

扩展点与职责分配

扩展位置	适合放什么	为什么不写进每个工具
执行前	权限、审批、Sandbox、参数策略	同一规则需要覆盖多类工具
执行包裹层	超时、重试、Metrics	需要覆盖完整执行时长
执行后	结果转换、敏感信息拦截	业务结果产生后才能判断
最终结果观察	审计、Trace、统计	只观察权威结果，不再改变它

本节小结

完整链路可以收束为四层

模型：根据上下文和工具 Schema 提出 Tool Call



协议：用 Input、Output、Error Schema 定义边界



Runtime：校验、授权、执行、超时、重试、审计和 Trace



Harness：保存 Tool Result，推动下一轮模型决策

其中真正需要 Agent 开发工程师完成的

- 1 把业务能力拆成边界清楚的工具；
- 2 写出模型能理解、程序能校验的协议；
- 3 从可信上下文注入身份和权限；
- 4 区分可重试错误与不可重复的副作用；
- 5 保证非法、越权和未审批调用永远不会执行；
- 6 让每次调用都能通过 Tool Result 和 Trace 被恢复、解释与审计；
- 7 用插件化方式装配工具和治理能力，同时保持 Agent Loop 稳定。